SelfImprove-Bench: Evaluating Generalization in Self-Improving Agents

Bangrui Xu[1,2,3*] Jie Yang[1,2,4*] Sidi Chen[1,2,3*] Xuanhe Zhou[1,2†] Jiantao Qiu[4]
Qianyu Yang[3] Zijie Wang[3] Fan Wu[1,2] Zhoufutu Wen[3†]
[1]Shanghai Jiao Tong University [2]Theseus Labs [3]Humanlaya [4]Shanghai AI Lab

Abstract

Recursively self-improving (RSI) systems are increasingly viewed as a promising means of accelerating progress toward artificial general intelligence. They aim to autonomously improve their capabilities by refining their working pipelines and recursively improving the strategies that guide subsequent refinements, with gains generalizing to unseen scenarios. However, existing evaluations lack reliable metrics to distinguish capability gains from scenario fitting, sustainable and extensible protocols for unseen tasks and difficulties, and fine-grained attribution of performance changes to improvement traces.

To this end, we propose **SelfImprove-Bench**, an RSI evaluation framework that organizes tasks into three primary capabilities (i.e., Exploring, Planning, Reasoning) as scenarios for guiding model evolution. We instantiated the tasks on games for controllability and reproducibility, and pairs them with scalable generation and orchestration driven by task templates and difficulty coefficients. Beyond task-level scores, SelfImprove-Bench organizes evaluation into *generalization ladder levels G0 to G4* that measure how far gains transfer from training conditions Experiments reveal that the evolutionary gains of existing RSI methods are inconsistent across generalization levels.

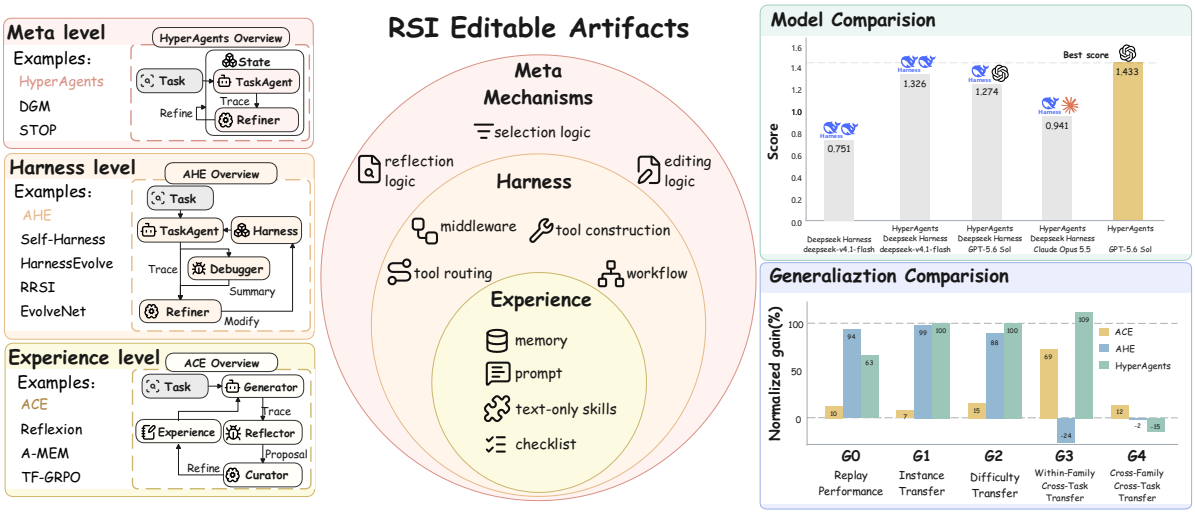


Figure 1 RSI at a glance. Left and center: representative RSI methods and editable artifacts. Top right: performance across different agent configurations. Bottom right: capability retention and generalization across G0–G4.

*Equal contribution.
†Corresponding author.

1 Introduction

Despite rapid advances in large language models (LLMs), conventional development still relies heavily on static datasets and offline training, limiting continuous adaptation to changing environments. Recursive self-improving (RSI) agents address this limitation by iteratively modifying their memories, skills, and harnesses using experience and feedback [1–3]. A central question is whether these improvements generalize beyond encountered instances to harder tasks, different task types, and ultimately distinct capabilities. Evaluating this progression is essential to understanding RSI as a path toward artificial general intelligence.

Existing benchmarks leave three challenges unresolved (Table 1). **(i) Systematic generalization assessment.** RSI benchmarks examine selected settings, such as cross-task or cross-difficulty transfer [4–6], but lack a unified taxonomy spanning instances, difficulty levels, tasks, and capabilities. **(ii) Scalable RSI evaluation.** Although some reinforcement learning benchmarks cover broader generalization settings [7], they lack a unified protocol for general-purpose RSI methods. Fixed task pools and limited difficulty ranges further constrain sustained evaluation as agents evolve and approach performance ceilings [8, 9]. **(iii) Fine-grained process diagnosis.** Existing analyses rely primarily on aggregate metrics, selected case studies, or persistence ablations [10, 11], leaving the relationship between evolution trajectories and generalization outcomes insufficiently understood.

To address these challenges, we introduce SelfImprove-Bench[1], an RSI evaluation framework combining capability-oriented tasks with a generalization ladder. (i) The ladder assesses progressively broader generalization across tasks targeting planning, reasoning, and exploration. It distinguishes five levels: repeated performance on the same instances (G0), unseen instances at the same difficulty (G1), new difficulty levels within the same task (G2), different tasks within the same capability category (G3), and tasks across capability categories (G4). (ii) A scalable task generation and orchestration module generates instances using task templates and difficulty coefficients, then assembles sequences with configurable task types, counts, difficulty levels, and replay frequencies. (iii) An agent-driven, fine-grained trace analysis module characterizes refinement semantics, modification scope, capability dimensions, and generalizability to reveal method-specific evolution patterns.

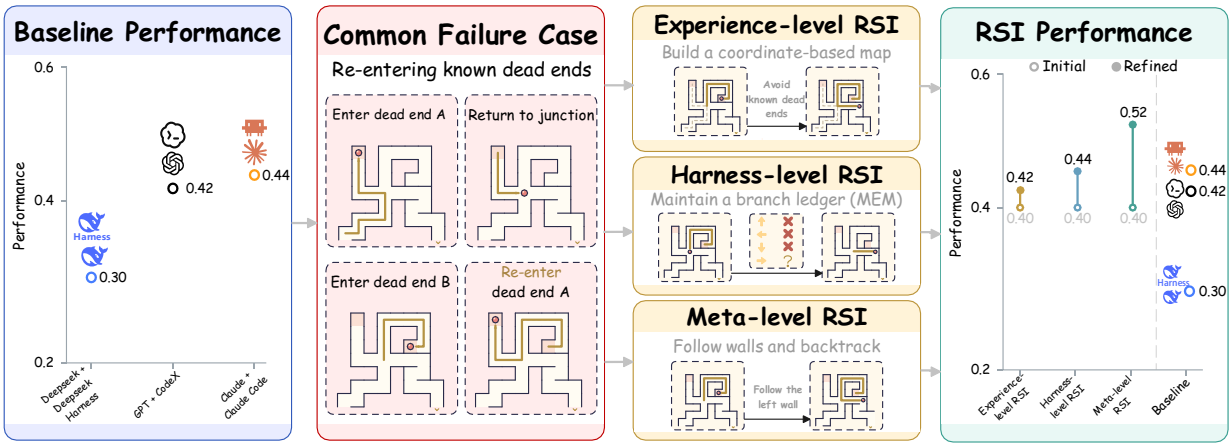


Figure 2 Example RSI-driven refinements. The blue panel shows the performance of different agent systems. The red panel illustrates a common failure case. The yellow panels show refinement examples from different RSI Methods. The green panel shows RSI performance before and after refinement

Using SelfImprove-Bench, we evaluate three representative classes of RSI methods and observe substantial variation across the generalization ladder, including ranking reversals on harder instances and different task types. Note we focus on non-training RSI methods, whose explicit modifications to prompts, memories, skills, and harnesses support a unified evaluation protocol and interpretable evolution traces. Their lower computational requirements also make repeated evaluation more tractable. Experiments across three rep-

[1]The code and data are available at https://github.com/theseus-labs-rsi/RSI-Bench

resentative classes of RSI methods reveal substantial differences along the generalization ladder, including ranking reversals on harder instances and different tasks. As illustrated in Figure 1, similar final scores can therefore conceal distinct evolution patterns and transfer ability.

Our main contributions are as follows.

(1) We introduce a capability-oriented generalization ladder for systematically evaluating RSI across instances, difficulty levels, tasks, and capabilities.

(2) We develop configurable task generation and orchestration to support scalable evaluation across the ladder.

(3) We evaluate representative RSI methods and analyze their evolution traces, revealing differences in improvement effectiveness, generalization, and modification patterns.

2 Related Work

2.1 RSI Methods

We classify RSI methods into three non-exclusive strategies based on their modification targets:

- **Experience-level RSI** adapts prompts, memories, and playbooks, as in Reflexion, A-MEM, and ACE [2, 19, 20].
- **Harness-level RSI** modifies tools, workflows, and executable code, as in SICA, Self-Harness, and AHE [3, 21, 22].
- **Meta-level RSI** refines candidate generation or selection, as in DGM's evolving self-modification agent and HyperAgents' editable meta-agent [1, 23].

Other strategies include reinforcement learning (RL), which typically updates policies using reward feedback within a predefined optimization framework [24]. Data synthesis provides another strategy, seeking to improve model capabilities through generated training data [25].

RSI methods flexibly integrate and organize these strategies into a continual self-improvement process and has been explored as a potential route toward artificial general intelligence (AGI) [26]. RSI evaluation should therefore examine whether improvements achieved through refinement transfer to harder tasks and unseen task types within and across capability categories.

2.2 RSI Relevant Benchmarks

RSI-Specific Generalization Evaluation. Common practices include cross-task testing without explicit capability categories organizing refinement and transfer [4, 27], and refinement and evaluation over mixed task streams [5, 6, 11, 28]. Mixed-stream evaluation examines cumulative performance and experience reuse, but alone does not isolate source-to-target transfer. Capability-aware protocols often focus on related-task or within-domain generalization [9, 12, 16]. Capability breakdowns may also serve post-hoc scoring rather than organize refinement and transfer [6]. Method-specific evaluations additionally demonstrate cross-domain transfer, as in ADAS [15].

Difficulty-based task organization does not necessarily include testing whether gains acquired on easier tasks transfer to harder ones [13]. Outside agent RSI, Reasoning Gym provides model-level evaluations of within-domain and cross-domain transfer, alongside difficulty-based evaluations [7]. A unified evaluation of agent RSI that combines capability-based refinement, difficulty generalization, and transfer across task categories remains underexplored.

RSI-Specific Trace Level Analysis. Process analysis commonly illustrates refinement through selected modification cases [18, 29]. Quantitative approaches assess experience contributions through persistence ablations and memory-free comparisons [8, 27], or track capability changes across refinement checkpoints [12]. Finer analyses can depend on specific artifact formats: SkillLearnBench measures skill quality, usage,

Work	Quantitative Analysis of RSI Modifications	Easy-to-Hard Transfer	Cross-Task Transfer	
			Within-Capability	Cross-Capability
EvoPathBench [12]	✓	—	—	—
EvoAgentBench [9]	—	—	✓	—
SEAGym [4]	—	—	✓	✓
SkillLearnBench [10]	✓	—	—	—
ServeLearnBench [13]	✓	—	—	—
PAST-Bench [8]	✓	—	—	—
STOP [14]	✓	—	—	—
HyperAgents [1]	—	✓	—	—
ADAS [15]	—	✓	✓	✓
RRSI [16]	—	—	✓	—
AI4AI-Bench [17]	✓	—	—	—
RSIBench-Data [18]	✓	—	—	—
Reasoning Gym [7]	—	✓	✓	✓
SelfImprove-Bench	✓	✓	✓	✓

Table 1 Comparison of evaluation coverage. Rows are grouped, from top to bottom, into RSI benchmarks, evaluations accompanying RSI methods, and related evaluations outside RSI. SelfImprove-Bench is shown separately. Task families are organized by agent ability. For task-type transfer, refinement uses type A, while type B is held out until evaluation; Same Ability and Cross Ability indicate whether the two types belong to the same ability-defined family. Easy-to-Hard requires evaluation on harder tasks than those used during refinement. Quantitative RSI Trace Analysis requires statistics over refinement trajectories rather than selected examples.

and alignment with execution trajectories, requiring explicit skill artifacts [10]. Related work also aggregates semantic categories of code modifications and relates them to performance, as in AI4AI-Bench [17]. However, statistical analysis jointly relating generalization performance, modification types, target capabilities, and the semantic content of individual changes remains underexplored.

Tested Tasks. Existing RSI evaluations involve tasks like software engineering, terminal interaction, web automation, expert-level reasoning [4, 30–33], or construct domain-specific tasks for training-algorithm design and financial workflows [6, 17]. However, end-to-end task scores offer limited capability-specific diagnosis, while fixed task pools may lose discriminative power as methods approach score ceilings.

3 SelfImprove-Bench

3.1 Evolution Protocol

Definition. Following the definition of RSI in [34], we call a refinement gain self-improvement to the extent that it persists on instances, difficulties, and tasks never exposed during refinement, and task fitting to the extent that it is confined to the training instances.

Task construction and orchestration. Each instance $\tau = G_g(d, \xi)$ is generated from game g, difficulty d , and seed ξ. The configuration θ first specify the games to include, then assigns difficulty levels and random seeds to generate their instances. It specifies how many instances enter the held-in and held-out sequences and in order, yielding $\mathcal{S}_{\text{in}} = (\tau_1, \ldots, \tau_N)$ for execution and refinement and $\mathcal{S}_{\text{out}} = (\tilde{\tau}_1, \ldots, \tilde{\tau}_M)$ for evaluation. Replay settings determine which held-in tasks can recur, how frequently, and how far back they are selected, counting only non-replay tasks. Replays do not trigger refinement. N includes replays. Detailed insertion rules are provided in Appendix A.2. The corresponding task collections $\mathcal{D}_{\text{in}}$ and $\mathcal{D}_{\text{out}}$ omit order and replay multiplicity. The generator constructs the held-out pool first and rejects duplicate task-descriptor hashes, excluding `task_id`, when generating non-replay held-in tasks (Appendix A.1). Game categorization is described in Section 3.2. Task configurations are provided in Appendix A.1.

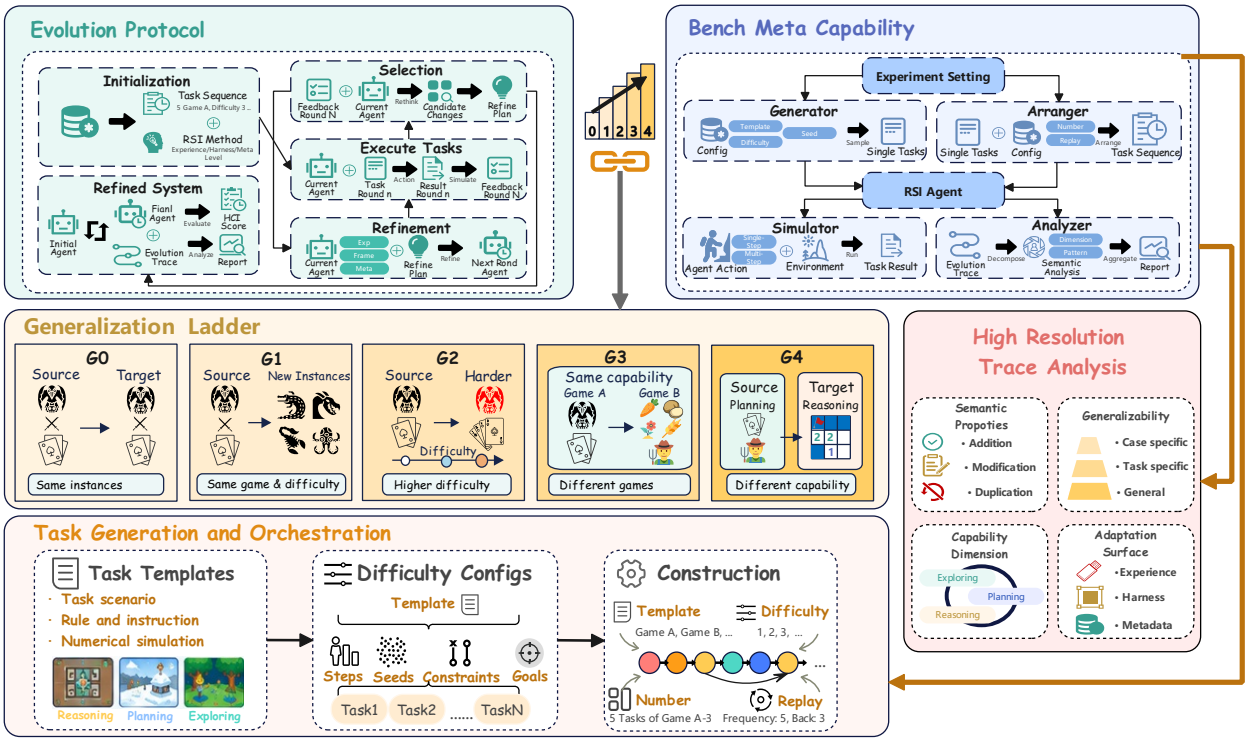


Figure 3 SelfImprove-Bench Overview. It supports (i) task generation and orchestration targeting primary agentic capabilities to support RSI refinement and (ii) G0–G4 generalization ladder evaluation.

We instantiate the framework on games, which satisfy the controllability and reproducibility requirements of RSI evaluation. Our game rules are explicit and state spaces are bounded, making it feasible to map planning, reasoning, and exploring onto different games and avoid the confounding of multiple abilities that complicates real-world tasks. Games inherently embody core characteristics of real-world problems such as partial observability, resource constraints, and multi-step decision making. Separating self-improvement from task fitting requires fresh instances and controlled difficulty, which games most readily provide. We use them as a controlled testbed that complements, rather than replaces, real-world RSI benchmarks. In addition, these games are agent tasks mainly based on text interaction and are related to the NLP community.

Agent initialization. For each RSI method, the initial task agent is $A_0 = (B, H_0)$, comprising a common backbone model B and a method-specific initial harness H_0. Backbone settings are reported in Section 4.1.

Refinement and selection. At step t, the current agent $A_{t-1} = (B, H_{t-1})$ executes $\tau_t \in \mathcal{S}_{\text{in}}$ through the game's action interface, producing an interaction trace and task score. Each task execution, except replays, is followed by refinement using benchmark-provided feedback. During refinement, the method may use tools to inspect and modify its designated workspace, but may not access SelfImprove-Bench modules or directly access, execute, or evaluate task instances. Feedback is supplied only through the prescribed interface (Appendix A.3). When the method proposes candidate harnesses, SelfImprove-Bench evaluates their agents on the same task instance τ_t and supplies the results, together with the current agent's result, to the method's selection procedure. Selection follows the method-specific rule and may retain the current harness. The resulting agent $A_t = (B, H_t)$ proceeds to the next task. Candidate validation uses held-in tasks only, without accessing $\mathcal{S}_{\text{out}}$. The backbone B remains fixed, and refinement ends when $\mathcal{S}_{\text{in}}$ is completed.

Evaluation and analysis. Evaluation uses saved system states and task results according to the protocols in Section 3.3. Process analysis extracts individual modification proposals from saved refinement reports and

edit records, then annotates and aggregates them using the criteria in Section 3.4. Annotation details are provided in Appendix A.4.

3.2 Task Categorization

For rule-constrained tasks with fixed action interfaces, we distinguish three interacting capabilities: **Exploration**, acquiring task-relevant information through observation and interaction. **Reasoning**, interpreting rules, deriving implications, and performing computations. **Planning**, organizing action sequences to achieve goals under constraints. Games are categorized by their primary capability demands, providing context for analyzing modification targets.

3.3 Generalization Ladder

The generalization ladder evaluates refinement gain, agent transfer, and refinement-mechanism effectiveness across G0–G4.

G0: Replay performance. For replay event j, let $A_{\text{init},j}$ denote the agent before its first execution of the replayed task, and $A_{\text{replay},j}$ the agent executing the current replay. At G0, compare $A_{\text{replay},j}$ with $A_{\text{init},j}$ on the replayed task τ_j.

G1–G4: Agent generalization. Relative to a source game g at difficulty d_0, $\mathcal{D}_1$ contains instances $G_g(d_0, \xi')$ and $\mathcal{D}_2$ contains $G_g(d_1, \xi')$ with $d_1 > d_0$. Collections $\mathcal{D}_1$ and $\mathcal{D}_2$ are drawn from $\mathcal{D}_{\text{out}}$, whose task descriptors are kept distinct from non-replay held-in descriptors by the construction protocol. Collections $\mathcal{D}_3$ and $\mathcal{D}_4$ contain other games g' not used for refinement, with $c(g') = c(g)$ and $c(g') \neq c(g)$, respectively. $c(g)$ denotes the primary capability category. For mixed-source sequences, source games and capability categories are specified for each transfer comparison. At G1–G4, compare A_T with A_0 on the same target instances.

Metrics. Across G0–G4, we use the HCI (Headroom-Closed Index) [34] to normalize gains relative to the initial gap to a reference score. We first average the baseline and comparison scores over target instances, then compute HCI from the resulting means. Let s_b and s_a denote these mean baseline and comparison scores and s_r the reference score :

$$\text{HCI} = \frac{s_a - s_b}{s_r - s_b}. \tag{1}$$

3.4 Semantic Process Analysis

Semantic relations. These indicate whether a proposal introduces new content, contradicts or revises existing proposals, or duplicates them, relative to the parent proposal snapshot and earlier proposals in the same stage.

Applicability scope. This distinguishes task-specific proposals, proposals applicable across tasks sharing a capability, and fully general proposals.

Functional targets. These identify which capability a proposal targets: exploration, reasoning, planning, or execution. Execution concerns operational mechanisms such as tool invocation, output formatting, retries, and error recovery.

Modification surfaces. These identify whether the change affects contextual artifacts, the execution harness, or the refinement mechanism itself.

		G0			G1			G2			G3		
Task	Method	Init.	Final	HCI (%)	Init.	Final	HCI (%)	Init.	Final	HCI (%)	Init.	Final	HCI (%)
Farm	ACE	1.078	1.067	-10.2	0.997	1.145	+81.3	0.784	0.859	+14.6	0.533	0.596	+7.0
	AHE	0.768	1.009	**+58.7**	1.105	1.080	-33.8	1.073	1.069	-1.8	0.674	0.792	**+15.5**
	Hyper	1.046	1.084	+28.0	1.048	1.179	**+100.0**	0.857	0.942	**+19.3**	0.597	0.596	-0.1
Combat	ACE	0.855	0.911	+9.7	0.533	0.592	+6.6	0.227	0.385	+14.5	0.997	1.123	+69.2
	AHE	1.065	1.410	**+93.8**	0.674	1.427	+99.2	0.301	1.196	+88.4	1.105	1.087	-24.3
	Hyper	0.881	1.226	+62.5	0.597	1.433	**+100.0**	0.233	1.314	**+100.0**	1.048	1.191	**+109.2**
Sweepmine	ACE	0.239	0.193	-6.0	0.134	0.316	+21.0	0.043	0.234	+20.0	0.094	0.067	-3.0
	AHE	0.432	0.420	-2.0	0.150	0.436	**+33.7**	0.107	0.350	**+27.2**	0.097	0.077	-2.2
	Hyper	0.193	0.216	**+2.8**	0.086	0.205	+12.9	0.014	0.117	+10.5	0.100	0.085	**-1.7**
Hero	ACE	0.169	0.124	-5.4	0.094	0.154	+6.7	0.017	0.124	+10.9	0.134	0.234	+11.5
	AHE	0.187	0.244	**+7.0**	0.097	0.526	**+47.5**	0.054	0.257	**+21.4**	0.150	0.318	**+19.8**
	Hyper	0.087	0.070	-1.9	0.100	0.085	-1.7	0.026	0.082	+5.7	0.086	0.264	+19.4
MC	ACE	0.162	0.077	-10.1	0.000	0.020	+2.0	0.000	0.350	+35.0	0.400	0.400	**+0.0**
	AHE	0.903	0.728	-179.4	0.505	0.720	**+43.4**	1.000	0.600	N/A	0.400	0.400	**+0.0**
	Hyper	0.278	0.483	**+28.4**	0.050	0.220	+17.9	0.000	0.400	**+40.0**	0.400	0.200	-33.3
Astray	ACE	0.318	0.395	+11.4	0.400	0.420	+3.3	0.222	0.211	-1.4	0.000	0.050	+5.0
	AHE	0.617	0.769	**+39.6**	0.400	0.440	+6.7	0.222	0.378	**+20.0**	0.505	0.550	**+9.1**
	Hyper	0.274	0.265	-1.2	0.400	0.520	**+20.0**	0.244	0.222	-2.9	0.050	0.000	-5.3

Table 2 Performance across G0–G3. Bold marks the highest available HCI for each game at each level, including ties. For G3, each row is the source game and Final is its agent's score on the paired target (Farm–Combat, Sweepmine–Hero, MC–Astray).

4 Experiments

We examine the gains from iterative refinement in ACE, AHE, and HyperAgents and whether these gains extend to unseen instances, harder tasks, and other games. We further analyze their refinement trajectories, task scheduling, and execution-harness refinement.

4.1 Experimental Setup

Tested Baselines. We evaluate ACE, AHE, and HyperAgents as representative methods of experience-level, harness-level, and meta-level RSI, respectively (Section 2). ACE maintains an evolving structured playbook; AHE revises the prompt, tools, and execution harness based on task feedback; and HyperAgents uses an improver to modify the task agent's code repository. We vary the backbone model, including GPT-5.6 Sol and Claude Opus 5.5, and the initial execution harness, including Codex and DeepSeek Harness, to assess how these choices affect RSI performance.

Configuration. Farm, Combat, Sweepmine, and Hero support difficulty levels 1–5. For MC and Astray, evaluation uses five fixed tasks per game from the official GameWorld [35] evaluation sets.

4.2 Main Results

We present the HCI scores of each RSI method at different levels (G0 to G4) in Tablee 2 and Figure 4.

(G0) Retention on Previously Encountered Instances. Refinement does not always preserve performance on previously encountered instances: replay scores fall below historical scores in 8 of the 18 method–task pairs. AHE on MC and ACE on Sweepmine improve on G1 but decline on G0.

(G1) Gains on Held-Out Instances. Most methods improve on unseen instances at the training difficulty: 16 of the 18 method–task pairs have positive HCI. HyperAgents has the highest HCI on Farm, Combat, and

Astray, and AHE on Sweepmine, Hero, and MC. A stronger initial agent does not necessarily stay ahead. On Farm, AHE starts above HyperAgents but finishes below it, with HCI of −33.8% and 100.0%.

(G2) Generalization to Harder Instances. Gains at the training difficulty do not necessarily extend to harder instances. ACE on Astray improves on G1 but declines on G2, and rankings can reverse: on Farm, HyperAgents beats AHE at the training difficulty but trails it on the Hard held-out set. On MC's Hard group, AHE falls from 1.000 to 0.600.

(G3) Within-Category Transfer. Within-category transfer can help or hurt, depending on direction. Among the Reasoning games, all three methods improve from Hero to Sweepmine but degrade in the reverse direction. An agent refined on another game of the same category can even outperform direct refinement on the target: HyperAgents reaches 109.2% HCI from Combat to Farm, above the 100.0% from refinement on Farm itself.

(G4) Cross-Category Transfer. Some HyperAgents improvements remain effective across capability categories, but not consistently: five of the eight directions in Figure 4 have positive HCI. The Farm-refined agent on Sweepmine and Hero, and the Combat-refined agent on Hero, exceed agents refined directly on those targets.

Overall, the evaluated methods show inconsistent refinement gains across generalization levels: improvements on unseen instances do not necessarily extend to higher difficulties or other tasks, and some settings exhibit performance degradation.

4.3 RSI Trajectory Analysis

The generalization ladder shows where gains persist; trajectory analysis examines what the methods change. We use GPT-5.6 Sol to split improvement artifacts into atomic opinions and annotate their semantic relations, applicability scope, functional targets, and modification surfaces. Table 3 covers the 915 opinions retained in the final snapshots of the six games.

Task-specific refinement can coexist with transferable gains. Task-specific opinions predominate in all three methods; category-general or general ones make up 14.2% of ACE's retained opinions, 7.8% of AHE's, and 36.5% of HyperAgents'. Yet G3 and G4 show positive transfer across games, sometimes exceeding direct refinement on the target.

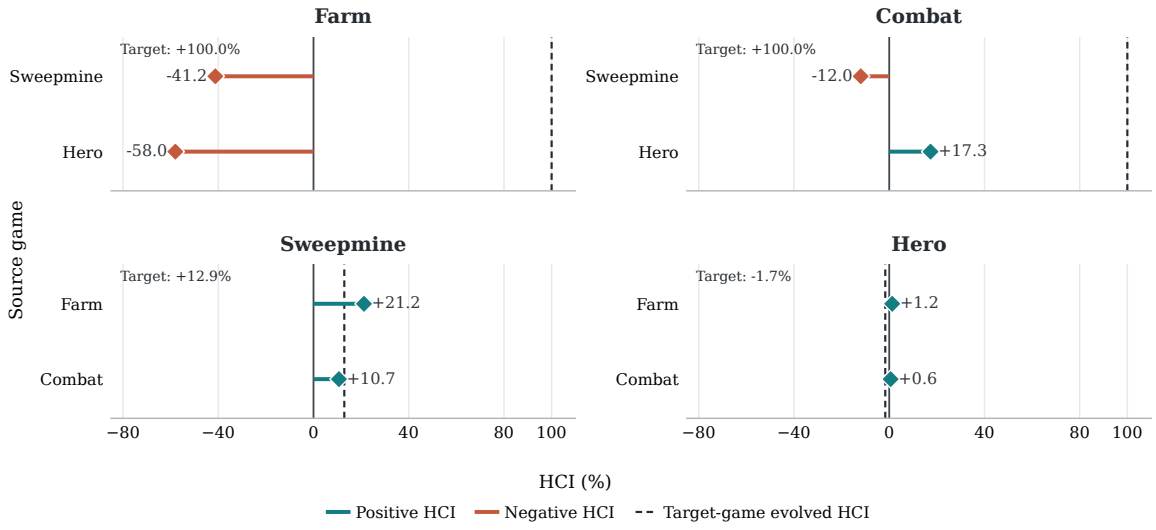


Figure 4 HyperAgents cross-category transfer (G4). Each panel is a target game; lines give the HCI of the agent refined on each source game. Dashed lines mark the HCI of refinement directly on the target.

		Records				Functional target (%)				Applicability scope (%)			Modification surface (%)		
Method	Game	All	Rev. (%)	Dup. (%)	N	Explor.	Reason.	Plan.	Exec.	Task Category		General	Prompt/Mem.	Harness	Meta
ACE	Farm	28	0.00	10.71	25	0.0	84.0	48.0	4.0	96.0	4.0	0.0	100.0	0.0	0.0
	Combat	51	3.92	17.65	40	22.5	95.0	52.5	7.5	100.0	0.0	0.0	100.0	0.0	0.0
	Hero	78	26.92	12.82	46	0.0	82.6	67.4	2.2	32.6	65.2	2.2	100.0	0.0	0.0
	Sweepmine	44	11.36	6.82	36	8.3	72.2	41.7	8.3	100.0	0.0	0.0	100.0	0.0	0.0
	MC	65	16.92	13.85	43	18.6	53.5	53.5	25.6	100.0	0.0	0.0	100.0	0.0	0.0
	Astray	66	12.12	12.12	35	17.1	65.7	25.7	11.4	100.0	0.0	0.0	100.0	0.0	0.0
AHE	Farm	91	32.97	13.19	50	8.0	32.0	8.0	68.0	76.0	18.0	6.0	38.0	62.0	0.0
	Combat	56	26.79	14.29	23	0.0	43.5	52.2	30.4	91.3	0.0	8.7	39.1	60.9	0.0
	Hero	397	21.16	46.10	89	16.9	82.0	27.0	25.8	91.0	4.5	4.5	98.9	1.1	0.0
	Sweepmine	99	27.27	18.18	53	5.7	50.9	41.5	26.4	100.0	0.0	0.0	71.7	28.3	0.0
	MC	210	40.48	15.71	82	17.1	24.4	82.9	4.9	95.1	2.4	2.4	98.8	1.2	0.0
	Astray	215	40.00	18.60	48	14.6	47.9	60.4	12.5	97.9	2.1	0.0	100.0	0.0	0.0
Hyper	Farm	434	55.53	17.05	102	2.9	77.5	56.9	12.7	42.2	46.1	11.8	96.1	3.9	0.0
	Combat	265	36.60	30.94	80	10.0	62.5	43.8	41.2	66.2	31.2	2.5	46.2	53.8	0.0
	Hero	383	18.80	67.62	47	0.0	57.4	42.6	6.4	80.9	14.9	4.3	100.0	0.0	0.0
	Sweepmine	386	38.08	43.52	57	14.0	64.9	50.9	8.8	59.6	29.8	10.5	100.0	0.0	0.0
	MC	141	40.43	36.88	22	22.7	54.5	81.8	45.5	81.8	9.1	9.1	95.5	4.5	0.0
	Astray	276	51.81	18.48	37	21.6	45.9	64.9	2.7	89.2	10.8	0.0	97.3	2.7	0.0

Table 3 Improvement statistics across six games. Recorded and retained opinions are compared across ACE, AHE, and HyperAgents by functional target, applicability scope, and modification surface. All/N: recorded/retained counts; Rev./Dup.: revision/duplicate rates (%).

Refinement emphasizes solving observed problems over exploring alternatives. Reasoning and planning appear in 61.2% and 49.6% of retained opinions, versus 11.0% for exploration. Current RSI methods may rely on observed trajectories to find and fix problems rather than explore alternatives.

Main-experiment changes concentrate on the task agent. 87.9% of retained opinions target prompts and memory and 12.1% the execution harness; none modify the improver or the improvement process.

More active revision may contribute to better refinement outcomes. ACE, AHE, and HyperAgents revise 14.2%, 30.6%, and 40.2% of their recorded opinions, respectively. AHE and HyperAgents revise existing opinions more frequently and generally achieve better final performance than ACE, suggesting that effective refinement may depend not only on adding new opinions but also on continually revising existing strategies.

Frequent revision does not eliminate duplicate opinions. HyperAgents has the highest revision rate, yet on Hero its duplicate rate reaches 67.6%, against 46.1% for AHE and 12.8% for ACE.

4.4 Ablation and Extended Analysis

4.4.1 Comparison with General-Purpose Agents

Figure 5 compares the refined agents with Codex, Claude Code, and DeepSeek Harness on Medium held-out tasks. Refined agents score highest on five of the six games: HyperAgents on Farm, Combat, and Astray, and AHE on Sweepmine and Hero. On MC, Claude Code outperforms all refined agents.

4.4.2 Effect of Task Scheduling

We evaluate four training task schedules using HyperAgents.

Random medium-difficulty training outperforms fixed-instance training on both Combat held-out sets and on Hard Hero tasks (Table 4): fixed-instance training scores 0.772 and 0.507 on Combat and 0.100 and 0.017 on Hero, against 1.433 and 1.314, and 0.085 and 0.082, for random medium-difficulty training. On Medium Hero tasks, however, fixed-instance training scores higher (0.100 versus 0.085).

Harder training instances do not necessarily help. Mixed and increasing difficulty score 1.195 and 1.080 on Hard Combat tasks, below the 1.314 of random medium-difficulty training, and 0.037 and 0.024 on Hard Hero tasks, below 0.082. Both also score lower on Medium Combat tasks, but higher on Medium Hero tasks (0.145

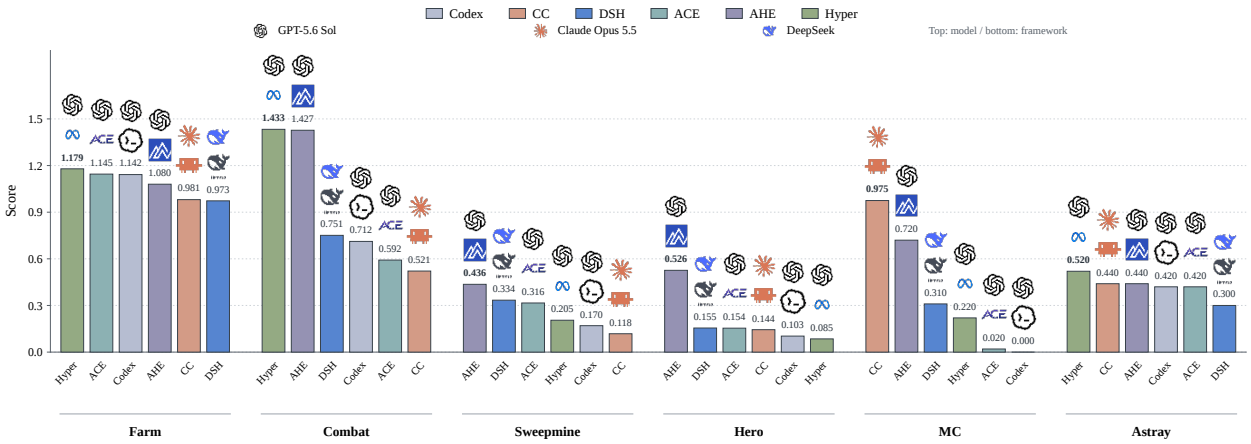


Figure 5 Comparison with general-purpose agents. Medium held-out scores of general-purpose agents and refined agents, ranked within each game.

	Combat		Hero	
Schedule	Medium	Hard	Medium	Hard
Random medium	**1.433**	**1.314**	0.085	**0.082**
Fixed medium instance	0.772	0.507	0.100	0.017
Mixed difficulty	1.388	1.195	0.145	0.037
Increasing difficulty	1.332	1.080	**0.152**	0.024

Table 4 Effect of training task scheduling. Medium and Hard held-out scores on Combat and Hero under random medium-difficulty, fixed-instance, mixed-difficulty, and increasing-difficulty training. Bold marks the highest score in each column.

and 0.152 versus 0.085), indicating that their benefits depend on the game and evaluation difficulty.

4.4.3 Exploratory Experiment: Directly Refining an Execution Harness

Task	Configuration	Score ↑
Combat	DeepSeek Harness	0.751
	HyperDSH (GPT-5.6 Sol)	1.274
	HyperDSH (Claude Opus 5.5)	0.941
	HyperDSH (DeepSeek)	1.326
	HyperDSH (DeepSeek, fixed improver)	1.207
	HyperAgents	**1.433**

Table 5 Direct execution-harness refinement. Medium held-out Combat scores of the original DeepSeek Harness, HyperDSH variants, and the main-experiment HyperAgents agent. HyperDSH is a variant that applies HyperAgent's improvement mechanisms to Deepseek Harness.

We evaluate HyperDSH's direct refinement of DeepSeek Harness on Combat, comparing it with the original DeepSeek Harness and the HyperAgents agent refined in the main experiment (Table 5). All three HyperDSH variants outperform DeepSeek Harness on Medium held-out tasks; the DeepSeek variant performs best among them at 1.326, below HyperAgents at 1.433. HyperDSH (DeepSeek, fixed improver), which keeps the improver unchanged while continuing to refine DeepSeek Harness, scores 1.207 on Medium, compared with 1.326 for HyperDSH (DeepSeek).

Despite permission to modify the entire repository, the main-experiment HyperAgents retains no Meta

Task	Configuration	All	N	Prompt/Mem. (%)	Harness (%)	Meta (%)
Combat	DeepSeek Harness	–	–	–	–	–
	HyperDSH (GPT-5.6 Sol)	107	44	79.5	0.0	20.5
	HyperDSH (Claude Opus 5.5)	57	31	54.8	0.0	45.2
	HyperDSH (DeepSeek)	127	37	59.5	0.0	40.5
	HyperDSH (DeepSeek, fixed improver)	85	27	96.3	3.7	0.0
	HyperAgents	265	80	46.2	53.8	0.0

Table 6 Trajectory comparison for execution-harness refinement. We compare HyperDSH variants with the main-experiment HyperAgents agent on Combat, reporting recorded and retained opinion counts and the modification-surface distribution of retained opinions. Configurations correspond to Table 5.

opinions, suggesting a limited tendency to modify its own improver spontaneously. In contrast, Table 6 shows that Meta accounts for 20.5–45.2% of retained opinions in HyperDSH variants that target DeepSeek Harness refinement and explicitly allow improver self-evolution, reflecting more modifications to the improver itself. Their retained modifications also shift toward the improver: none of the three variants retains Harness-level opinions, compared with 53.8% for HyperAgents. The three HyperDSH variants also produced fewer opinions: 57–127 recorded occurrences, compared with 265 for HyperAgents, indicating less active opinion generation when refining an existing execution harness.

5 Conclusion

We presented SelfImprove-Bench, a scalable benchmark for RSI evaluation. Different from conventional benchmarks, SelfImprove-Bench supports template- and difficulty-controlled task generation for configurable evaluation. We propose a generalization ladder to reflect the transfer to new instances, new difficulties, new tasks within a capability, and across capabilities. We evaluate three representative classes of RSI methods and find that their effectiveness varies across the generalization ladder, with their relative rankings reversing when transferring to tasks of higher difficulty or different types.

References

[1] J. Zhang, B. Zhao, W. Yang, J. Foerster, J. Clune, M. Jiang, S. Devlin, and T. Shavrina, "Hyperagents," arXiv preprint arXiv:2603.19461, 2026. [Online]. Available: https://arxiv.org/abs/2603.19461

[2] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun, "Agentic context engineering: Evolving contexts for self-improving language models," arXiv preprint arXiv:2510.04618, 2025. [Online]. Available: https://arxiv.org/abs/2510.04618

[3] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," arXiv preprint arXiv:2604.25850, 2026. [Online]. Available: https://arxiv.org/abs/2604.25850

[4] C. Zheng, C. Xue, B. Liang, J. Yang, and C. Zhang, "Seagym: An evaluation environment for self-evolving llm agents," arXiv preprint arXiv:2606.17546, 2026.

[5] L. Huang, C. Yang, H. Zhou, H. Song, Z. Chen, R. Le, Y. Song, W. X. Zhao, and T. Zhang, "Evo-Bench: Can language models improve agent harness?" arXiv preprint arXiv:2608.09096, 2026. [Online]. Available: https://arxiv.org/abs/2608.09096v1

[6] B. Deng, K. Zhou, L. Guo, C. Tao, X. Chen, C. Xie, R. Liang, F. Chen, and C. Zhang, "FinEvo-Bench: A longitudinal benchmark for self-evolving agents in professional financial workflows," arXiv preprint arXiv:2608.06144, 2026. [Online]. Available: https://arxiv.org/abs/2608.06144

[7] Z. Stojanovski, O. Stanley, J. Sharratt, R. Jones, A. Adefioye, J. Kaddour, and A. Köpf, "Reasoning Gym: Reasoning environments for reinforcement learning with verifiable rewards," in Advances in Neural Information Processing Systems, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/528d56195a2c77c808494c86fa7c77ad-Abstract-Datasets_and_Benchmarks_Track.html

[8] S. Xue, Z. Ding, Y. Shen, Y. Wang, Z. Yin, Y. Wu, Y. Chen, M. Wang, and L. Yang, "Past-bench: Benchmarking the foundations of recursive self-improvement in personal agents," arXiv preprint arXiv:2608.04003, 2026.

[9] X. Gao, C. Hu, H. Chen, P. Yao, Z. Wang, Y. Bai, Z. Wu, Y. Han, X. Cong, J. Gui, Y. Deng, and T. Li, "EvoAgentBench: Benchmarking agent self-evolution via ability transfer," arXiv preprint arXiv:2607.05202, 2026. [Online]. Available: https://arxiv.org/abs/2607.05202

[10] S. Zhong, Y. Lu, J. Ning, Y. Wan, L. Feng, Y. Ao, L. F. R. Ribeiro, M. Dreyer, S. Ammirati, and C. Xiong, "SkillLearnBench: Benchmarking continual learning methods for agent skill generation on real-world tasks," arXiv preprint arXiv:2604.20087, 2026. [Online]. Available: https://arxiv.org/abs/2604.20087v1

[11] J. Zheng, X. Cai, Q. Li, D. Zhang, Z. Li, Y. Zhang, L. Song, and Q. Ma, "LifelongAgentBench: Evaluating LLM agents as lifelong learners," arXiv preprint arXiv:2505.11942, 2025. [Online]. Available: https://arxiv.org/abs/2505.11942

[12] H. Lin, C. Liu, X. Bai, X. Jin, Y. Li, N. Zheng, and X. Cao, "Beyond endpoint performance: Process-level evaluation of self-evolving agents," arXiv preprint arXiv:2609.24663, 2026. [Online]. Available: https://arxiv.org/abs/2609.24663

[13] H. Zheng, Y. Di, R. Sadhukhan, S. Jin, and B. Chen, "ServeLearnBench: How well can agents self-improve from serving experience?" arXiv preprint arXiv:2610.07792, 2026. [Online]. Available: https://arxiv.org/abs/2610.07792

[14] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, "Self-taught optimizer (STOP): Recursively self-improving code generation," in First Conference on Language Modeling, 2024. [Online]. Available: https://arxiv.org/abs/2310.02304v3

[15] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in The Thirteenth International Conference on Learning Representations, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/36b7acf6f6010652b3f2a433774a66fe-Abstract-Conference.html

[16] P. Xia, R. Han, Z. Wang, Y. Chen, Y. Zhang, Y. Lee, C. Huang, H. Yu, Z. CuiZhu, Y. Ming, H. Yao, B. Gokturk, T. Pfister, and C.-Y. Lee, "RRSI: Regularized recursive self-improvement of agent harnesses," arXiv preprint arXiv:2609.24972, 2026. [Online]. Available: https://arxiv.org/abs/2609.24972v1

[17] Y. Chi, W. Li, D. Hong, X. Wang, M. Gao, K. Yang, B. He, Y. Zheng, C. Xiao, and Q. Na, "Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement," arXiv preprint arXiv:2608.20318, 2026.

[18] F. Meng, L. Du, Q. Chen, Z. Zhao, H. Lu, M. Hu, and M. Q. Shieh, "RSIBench-Data: Benchmarking data-centric research for recursive self-improvement," arXiv preprint arXiv:2607.25886, 2026. [Online]. Available: https://arxiv.org/abs/2607.25886

[19] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," arXiv preprint arXiv:2303.11366, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[20] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic memory for LLM agents," arXiv preprint arXiv:2502.12110, 2025. [Online]. Available: https://arxiv.org/abs/2502.12110

[21] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," arXiv preprint arXiv:2504.15228, 2025.

[22] H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu, "Self-harness: Harnesses that improve themselves," arXiv preprint arXiv:2606.09498, 2026. [Online]. Available: https://arxiv.org/abs/2606.09498

[23] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin gödel machine: Open-ended evolution of self-improving agents," in International Conference on Learning Representations, vol. 2026, 2026, pp. 104 223–104 294.

[24] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," arXiv preprint arXiv:2203.02155, 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[25] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," arXiv preprint arXiv:2203.14465, 2022. [Online]. Available: https://arxiv.org/abs/2203.14465

[26] E. Nivel, K. R. Thórisson, B. R. Steunebrink, H. Dindo, G. Pezzulo, M. Rodriguez, C. Hernandez, D. Ognibene, J. Schmidhuber, R. Sanz, H. P. Helgason, A. Chella, and G. K. Jonsson, "Bounded recursive self-improvement," arXiv preprint arXiv:1312.6764, 2013. [Online]. Available: https://arxiv.org/abs/1312.6764

[27] Y. Wang, Z. Zhang, M. Chi, K. Yu, Y. Li, M. Peng, B. Tong, C. Zhang, Y. Zhou, and J. Li, "EvoMemBench: Benchmarking agent memory from a self-evolving perspective," arXiv preprint arXiv:2605.18421, 2026. [Online]. Available: https://arxiv.org/abs/2605.18421v2

[28] T. Guan, Y. Wang, H. Yang, S. Cao, S. Liu, Y. Hu, J. Li, and M. Zhang, "ContinualSkillBench: Can LLM agents truly evolve their capabilities?" arXiv preprint arXiv:2608.03874, 2026. [Online]. Available: https://arxiv.org/abs/2608.03874

[29] P. Asawa, C. M. Glaze, G. Orlanski, R. Ramakrishnan, B. Xu, A. Biswal, V. S. Chen, F. Sala, M. Zaharia, and J. E. Gonzalez, "Continual Learning Bench: Evaluating frontier AI systems in real-world stateful environments," arXiv preprint arXiv:2606.05661, 2026. [Online]. Available: https://arxiv.org/abs/2606.05661

[30] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" arXiv preprint arXiv:2310.06770, 2023. [Online]. Available: https://arxiv.org/abs/2310.06770

[31] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan et al., "Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces," arXiv preprint arXiv:2601.11868, 2026. [Online]. Available: https://arxiv.org/abs/2601.11868

[32] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," arXiv preprint arXiv:2307.13854, 2023. [Online]. Available: https://arxiv.org/abs/2307.13854

[33] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi et al., "Humanity's last exam," arXiv preprint arXiv:2501.14249, 2025. [Online]. Available: https://arxiv.org/abs/2501.14249

[34] Y. Duan, Y. Liu, Z. Tang, H. Chen, J. Zhou, Y. Liu, B. Xu, Y. Wu, S. Chen, Y. Zhou, H. Wang, X. Yu, S. Han, X. Zhu, L. Zhou, B. Lu, W. Zhou, J. Liu, N. Fang, J. Tian, R. Chen, Y. Li, K. Zuo, K. Zhang, Q. Yang, Z. Wang, J. Qiu, C. He, G. Li, B. Zhou, Z. Liu, Z. Wen, J. Kang, X. Zhou, and F. Wu, "The last ai built by humans: Toward genuine recursive self-improvement," 2026. [Online]. Available: https://arxiv.org/abs/2609.11873

[35] M. Ouyang, S. Hu, K. Q. Lin, H. T. Ng, and M. Z. Shou, "GameWorld: Towards standardized and verifiable evaluation of multimodal game agents," arXiv preprint arXiv:2604.07429, 2026. [Online]. Available: https://arxiv.org/abs/2604.07429

[36] A. A. Zheng, K. Misra, D. I. Beaver, and J. J. Li, "Hero's journey: Testing complex rule induction with text games," arXiv preprint arXiv:2606.02556, 2026.

A Evaluation Protocol Details

A.1 Task Configurations

The task-related fields of θ determine which instances are generated and how they form $\mathcal{S}_{\text{in}}$ and $\mathcal{S}_{\text{out}}$; model and runtime settings are configured separately. Game definitions, difficulty parameters, and scoring rules are provided in Appendix B.

Held-in tasks. The ordered `schedule` list contains blocks with `game`, `difficulty`, and `count` fields. Each block generates the requested number of non-replay instances of that game and difficulty. Blocks are concatenated in list order before replay insertion; `count` excludes inserted replays. The block-level `replay` flag controls eligibility for replay insertion, whose exact semantics are defined in Appendix A.2. The standard ordered setting uses `random=false` and `repeat=false`; these are also the defaults.

Held-out tasks. The ordered `holdout` list specifies evaluation blocks using the same `game`, `difficulty`, and `count` fields. For each requested game–difficulty pair, `holdout_tasks_per_difficulty` sets the candidate pool size, while the block's `count` selects that many instances from the beginning of its pool. Selected blocks form $\mathcal{S}_{\text{out}}$ in list order, without replay insertion.

Seeds and reconstruction. The `seed` and `holdout_seed` fields control held-in and held-out generation, respectively. For newly generated instances, task seeds are derived deterministically from the root seed, phase, game, difficulty, instance index, and generation attempt using SHA-256. The current generator rejects duplicate task-payload hashes, excluding `task_id`, across the generated held-out pool and non-replay held-in instances; replay insertion deliberately reuses an existing task. This is descriptor-level deduplication: seed fields remain in the hashed payload, so different descriptors are not guaranteed to produce different underlying

game states. Each run saves its resolved configuration and task manifest as `config.resolved.json` and `task_manifest.json`, including task instances, seeds, sequence order, and replay links. These artifacts and the corresponding generator version specify the realized sequences.

A.2 Replay Protocol

Execution. Each replay executes the original task instance using the current agent, and records the resulting score for G0 evaluation. Replay executions do not trigger refinement.

Insertion rule. Replays are inserted while constructing the held-in sequence, before execution. Let $u_1, \ldots, u_n$ be the non-replay tasks placed so far, with u_n belonging to the current schedule block. After appending u_n, insert a replay of $u_{n-\ell}$ if the current block enables replay, the interval e and lag ℓ are positive, $n \bmod e = 0$, and $n > \ell$. Both interval and lag count only non-replay tasks, globally across schedule blocks.

Boundary conditions. No replay is inserted when history is insufficient ($n \leq \ell$), when the current block disables replay, or when replay is globally disabled ($e = \ell = 0$). No additional replay is inserted to complete a partial interval. The rule also applies after the final non-replay task: a replay is appended if the same conditions hold. The ordering above assumes `random=false`; enabling `random` shuffles the assembled sequence, including its replay entries, after insertion.

A.3 Feedback Specification

Abstract interface. The benchmark passes task execution results to refinement through a `FeedbackResult` object,

$$\mathcal{F} = (\text{id}, \text{task}, \text{trajectory}, \text{evaluation}).$$

The identifier links feedback to its execution, and `task` contains the game-specific task descriptor. The ordered `trajectory` contains step records with `step_index`, `prompt`, `image_records`, `action`, `environment_response`, `success`, `score`, `feedback`, and an optional `agent_call`. The latter retains the model response and, when available, reasoning, token usage, provider, model, and call metadata. Image records reference available images; text-only interactions use an empty image list. Step scores, success flags, and feedback may be absent or null when the game does not provide them.

The `evaluation` object contains execution status, task success, the task score, optional explanatory feedback, and game-specific metrics. Execution status distinguishes completion, error, timeout, and cancellation; a completed execution need not solve the task successfully. Metrics depend on the game and can include consumed actions or restart counts. Game-specific scoring is described in Appendix B.

A.4 Annotation Setup and Validation

The analysis records use `gpt-5.6-sol` for semantic annotation. We checked 100 pre-extracted proposals, stratified by game and method across six games and three methods, against their source modifications. Table 7 reports agreement with human labels after human-approved, assistant-assisted review with access to machine labels; these are post-review results rather than independent blind agreement estimates.

Scope and Surface each take one label. Function permits multiple labels because a single proposal can affect several functions, such as reasoning and planning. Complete-set agreement is therefore stricter than any-label overlap, which requires only $H_i \cap M_i \neq \varnothing$ for human and machine label sets H_i and M_i. Mean Jaccard similarity averages $|H_i \cap M_i|/|H_i \cup M_i|$ across proposals, accounting for both missing and additional labels. The results indicate strong correspondence after review, particularly for Surface, while differences remain in Scope and complete Function combinations. The 96% Function overlap indicates a shared functional label on most proposals, not complete agreement or classification accuracy.

Measure	After review
Scope exact agreement	88%
Surface exact agreement	99%
Function any-label overlap	96%
Function mean Jaccard	0.828

Table 7 Agreement between fixed machine labels and reviewed human labels on 100 sampled proposals.

B Game Design

In this appendix, we describe the game design in our dataset and detail how game instances are generated by combining task templates with difficulty coefficients.

B.1 Planning Games

B.1.1 Task Definition

Farm. In this game, an agent operates a farm under cash, land, and time constraints. Each task specifies the initial cash, number of plots, planning days horizon, and a task-specific crop catalog containing seed prices, maturation times, harvest yields, and selling prices. The agent receives the entire instance in one non-interactive call and returns a JSON plan indexed by day. Within each day, actions are executed in the fixed order buy, plant, harvest, sell. The objective is to maximize terminal cash profit, i.e., final cash minus initial cash.

Combat. Combat is a deterministic deck-building game comprising alternating combats and shops followed by a boss encounter. Player health, gold, and deck composition persist across steps. At a combat step, the agent observes its current state, owned cards, the enemy's repeating public intent cycle, and the exact deterministic hands. It returns the cards to play on selected turns subject to hand multiplicity and energy constraints. At a shop step, the agent may buy each offered card and purchase one heal, all within its current gold budget. The goal is to defeat the complete encounter chain, including the boss, while maximizing an objective that rewards defeated enemies, remaining health, and remaining gold.

B.1.2 Task Generation

Farm. Let d denote difficulty. The generator fixes initial cash to 200, samples $2d+2$ crops uniformly without replacement from the global catalog, and sets both the number of plots and the days horizon to $4d+4$. Thus, increasing d simultaneously enlarges the action vocabulary, land capacity, and temporal planning depth. Crops are not tiered, and instance variation arises from the sampled combinations of crop prices, yields, and maturation times.

Combat. For difficulty $d \in \{1,2,3,4,5\}$, the available catalog contains every card of tier at most d. The initial deck contains a fixed card set, plus one uniformly sampled card from each tier $2,\ldots,d$. Player parameters remain fixed at 80 maximum and initial health, 120 initial gold, 3 energy, and 5 drawn cards per turn. Each combat permits at most 15 turns. For every tier $i \le d$, the generator samples one non-boss enemy of tier i as the enemy, and then appends a shop. Each shop offers three distinct cards. One offer is forced to have tier $\min(d, i+1)$, while the other two are sampled from cards at or below that tier. A tier-d boss is at last, producing $d+1$ combats and $2d+1$ total steps. Enemies and cards become stronger with tier, while the longer step chain increases resource coupling across decisions.

B.1.3 References Calculation

Planning a game involves complex permutations and combinations, and it is difficult to directly compute an optimal solution for it. Therefore, specific search algorithms are used to find a near-optimal solution within a

limited time as a pseudo ground-truth.

Farm. The pseudo ground-truth for each farm game is derived from a rolling search. On each day, matured batches are first harvested and sold. Then for each candidate crop, the heuristic tries to greedily plant it into as many free plots as cash permits, and rolls the resulting state to the days horizon using the same daily greedy rule. The candidate with the largest projected terminal cash is committed, and the pseudo ground-truth profit is the terminal cash minus the initial cash. In the above search branches, the daily decision is to plant as much of the same crop as possible, whereas in fact there may be better strategies such as planting a combination of multiple crops. Therefore, the ratio (score) of these strategies compared with the pseudo ground-truth can exceed 1.

Combat. Combat computes its reference in two search levels. For each combat stage, the solver tries every legal card combination by turns and simulates its outcome. Among winning plans, the solver prefers higher remaining health and then fewer turns. At each shop, it tries every affordable combination of the three offered cards, both with and without healing. The solver then carries these alternatives through the following stages. To keep the search tractable, partial runs are ranked using remaining health, gold, and the damage and block provided by the deck, and only the top-k plans are continued. After the boss, the completed run with the highest official objective becomes the reference candidate.

B.2 Reasoning Games

B.2.1 Task Definition

Hero. We adapt Hero's Journey [36], a text-based adventure benchmark for hidden-rule induction. Given demonstrations and a task description, the agent infers the missing rules and returns a complete action sequence in one response. The objective is to reach the task goal within the action budget.

Each benchmark task contains five separately initialized episodes with different seeds. A successful episode is scored by the ratio of reference solution length to executed action count; an unsuccessful episode receives zero. The task score is the mean episode score.

Sweepmine. The agent identifies mines on a partially observed board. Each revealed safe cell indicates the number of mines among its eight neighboring cells. At each step, the agent opens, marks, or unmarks a cell and receives the updated board. The objective is to mark every mine without marking any safe cell within the action budget. Opening a mine generates a new board without resetting the remaining budget.

An unsuccessful task receives zero. For a successful task, the score is $(B - a)/(B - m)$, where B is the action budget, a the number of actions consumed, and m the number of mines.

Game	d1	d2	d3	d4	d5
Hero	Ovr/1/3	Ord/2/2	Ord/3/2	Ord/4/2	Ord/5/2
Sweepmine	5×5/ 3/16	6×6/ 5/22	7×7/ 8/30	8×8/ 11/40	9×9/ 15/52

Table 8 Reasoning game difficulty configurations. Hero entries give rule template / hidden action blocks / budget multiplier; Sweepmine entries give board size / mines / action budget.

B.2.2 Task Generation

Hero. We generate five episodes per task using different seeds, with templates and element pools adapted from the official implementation. Table 8 specifies the rule template, hidden block count, and action-budget multiplier. Override (Ovr) uses class-based defaults with a role-based exception; Ordered (Ord) requires inferring each block's action type, repetition count, and relative order. The action budget is the reference solution length times the multiplier.

Sweepmine. We generate boards with seed-controlled mine placement. Difficulty-specific configurations specify the board size as rows $\times$ columns, the total number of mines, and the action budget as the maximum

number of attempted actions across all board restarts (Table 8). The game rules and action interface remain fixed across levels.

B.3 Exploring Games

B.3.1 Task Definition

MC. Minecraft (MC) is a GameWorld browser-based first-person resource-collection and environment-exploration task. The agent starts with an empty inventory and uses visual observations to locate target resources, move, adjust its view, and interact with blocks to complete a collection goal within the step budget. Each step provides the task description, a short interaction history, and the current screenshot, and the agent returns one JSON action.

Astray. Astray is a GameWorld maze-navigation task in which the agent uses screenshots to guide a ball to the exit. We adapt the scoring to use the maximum relative reduction in breadth-first search (BFS) shortest-path distance to the exit over the run.

B.3.2 Task Generation

MC. We define 20 training tasks with different collection goals and target quantities; evaluation uses the five fixed MC tasks from the official GameWorld evaluation set.

Astray. Training mazes have side length $L(d) = 2d + 9$ for difficulty $d \in \{1, \ldots, 5\}$, with layouts determined by random seeds. Evaluation uses the five fixed Astray tasks from the official GameWorld evaluation set.

C Failure Patterns and RSI Refinement Examples

Table 9 presents failure patterns observed in both baseline systems and initial agent systems of evaluated RSI methods, along with representative refinements proposed by each evaluated RSI method to address these failures.

Table 9 Failure patterns and example refinements produced by ACE, AHE, HyperAgents, and HyperDSH. Each pattern was observed in at least one evaluated baseline's traces. Red highlights flag flawed assumptions or rules, green highlights flag concrete strategies targeting the stated failure pattern and unhighlighted text presents general refinement guidance.

Game	Failure pattern	ACE refinement	AHE refinement	HyperAgents refinement	HyperDSH refinement
Hero	Copying demonstrated plans, causing redundant actions.	**[Experience]** Decompose demonstrations and combine only action blocks needed for the target attributes.	**[Experience]** Recombine blocks by target attribute, retaining contributions from unchanged attributes and replacing others only with contrastive evidence.	**[Experience]** Model each feature as an action block and combine the blocks inferred for the target. [Error: No guaranteed one-to-one attribute–block mapping.]	**[Experience]** Select action blocks by target attributes to build the primary plan and allow up to two additional rounds of blocks drawn from other demonstrations. [Correct: Limits retries while allowing alternative action blocks.]
Hero	Misinterpreting action types, order, or counts in demonstrations.	**[Experience]** Infer action blocks through controlled comparisons between demonstrations.	**[Harness]** Guide the agent to construct and run shell-based analyses that extract demonstrations and track evidence by attribute.	**[Experience]** Infer action types and counts separately.	**[Experience]** Select action blocks with their counts by target attribute, then check their order and placement against demonstrations.

Game	Failure pattern	ACE refinement	AHE refinement	HyperAgents refinement	HyperDSH refinement
Sweepmine	Marking cells incorrectly without checking all current board constraints.	**[Experience]** Check all local constraints across the board and recompute them after each action.	**[Experience]** Use stepwise prompts to rebuild board constraints and incorporate the total mine-count constraint.	**[Experience]** Derive residual constraints from the current board and combine overlapping constraints with the total mine count.	**[Experience]** Use a checklist to check local constraints. A skill also instructs the agent to track feasible mine placements and use the total mine count to guide inference.
MC	Misidentifying collection targets or whether they can be collected.	**[Experience]** Verify that the target can be mined.	**[Harness]** Verify the target's current state. After a failed mining attempt, try a different target face. [Correct: A concrete re-aiming policy after mining failure.]	**[Experience]** Verify the target's presence, reachability, and type.	**[Experience]** Check target type and state before collecting, and additionally monitor collection progress.
MC	Failing to adapt when navigation makes no progress.	**[Experience]** Compare position and distance to the resource before and after movement to assess progress, then reorient or excavate if movement is ineffective.	**[Harness]** Switch to excavation toward the resource when exploration stalls.	**[Harness]** Prevent consecutive repeated actions and turn or jump when movement stalls. [Correct: Enforces an action-repeat limit in code.]	**[Experience]** Compare the current position with the resource's location and stop moving further when the agent is far from it.
Astray	Misjudging movement direction or traversable paths.	**[Experience]** Assess movement using velocity and displacement along the commanded axis.	**[Experience]** Identify paths from connected floor regions and follow confirmed bends without repeatedly changing direction.	**[Experience]** Assess movement using nearby walls and displacement along the commanded axis.	**[Experience]** Assess movement using structured coordinate changes; use changes in position relative to landmarks instead when coordinate data are unavailable.
Astray	Confusing explored routes and known dead ends with unexplored branches.	**[Experience]** Record exploration history and connectivity to build a map.	**[Experience]** Carry forward a compact MEM ledger of explored branches and confirmed dead ends in each reasoning response.	**[Experience]** Add explicit wall-following and backtracking rules. [Correct: A consistent strategy for systematic maze exploration.]	**[Experience]** Follow a wall on a fixed side, record tried routes and switch to an untried route after consecutive moves away from the goal. [Correct: A consistent strategy for systematic maze exploration.]
Combat	Prioritizing victory or fast kills over remaining HP.	**[Experience]** Compare combat plans by remaining HP.	**[Harness]** Build a dynamic programming solver to maximize remaining HP after victory. [Correct: Implements HP-oriented search in code.]	**[Harness]** Build a solver to compare complete combat plans and maximize remaining HP after victory. [Correct: Implements HP-oriented search in code.]	**[Experience]** Use text instructions to compare combat plans through dynamic programming and maximize remaining HP. [Correct: Specifies HP-oriented dynamic programming.]

D Additional Results Table

Table 10 Main results (%) on DeepSWE, SWE-EVO, and SWE-Milestone. Full-benchmark results are reported with Python subsets in parentheses; SWE-EVO is Python-only. **Bold** denotes MemTrace, and underlining denotes the best baseline. Δ_{best} reports the percentage-point difference from the best baseline, computed directly from the reported values.

	DeepSWE	**SWE-EVO**			**SWE-Milestone**			
Method	pass@1	Resolved	Fix	Apply	Score	Prec.	Recall	Resolved
Harness: Codex CLI								
Native	35.4 (32.4)	47.8	53.4	97.8	28.6 (30.7)	26.9 (26.4)	41.9 (84.5)	7.1 (8.3)
Obs. Masking	34.5 (38.2)	65.2	72.1	97.8	24.4 (30.0)	24.2 (26.0)	37.1 (84.9)	9.2 (8.3)
RepoGraph	– (41.2)	58.7	68.8	97.8	– (45.2)	– (36.7)	– (91.5)	– (8.3)
MemTrace	**56.6 (67.6)**	**69.6**	**70.6**	**100.0**	**46.4 (46.1)**	**43.6 (37.8)**	**60.9 (91.7)**	**16.3 (16.7)**
Δ_{best}	+21.2 (+26.4)	+4.4	-1.5	+2.2	+17.8 (+0.9)	+16.7 (+1.1)	+19.0 (+0.2)	+7.1 (+8.4)
Harness: mini-swe-agent								
Native	48.7 (47.1)	52.2	54.1	95.7	34.5 (17.6)	34.7 (19.6)	45.6 (54.2)	11.2 (0.0)
Obs. Masking	26.6 (20.6)	23.9	33.6	97.8	24.9 (21.9)	25.1 (21.4)	29.9 (22.5)	8.2 (8.3)
RepoGraph	– (32.4)	30.4	37.9	97.8	– (30.8)	– (25.5)	– (69.1)	– (0.0)
MemTrace	**66.4 (64.7)**	**63.0**	**68.7**	**93.5**	**48.3 (37.3)**	**46.9 (32.6)**	**71.2 (82.2)**	**19.4 (16.7)**
Δ_{best}	+17.7 (+17.6)	+10.8	+14.6	-4.3	+13.8 (+6.5)	+12.2 (+7.1)	+25.6 (+13.1)	+8.2 (+8.4)